

PROJECT DOCUMENTATION

AI WEATHERWISE API

1. INTRODUCTION :

❖ **PROJECT TITLE:** AI WEATHERWISE API

❖ **TEAM LEADER:**

Name: Kaviya.G

Mail Id: kaviyamalliga2007@gmail.com

Nm Id: 9C30D4292A6E3C994324E5CC028E2D00

❖ **TEAM MEMBERS:**

Name: Jenifer.D

Mail Id: jeniferdhanraj13@gmail.com

Nm Id: 0CD653BB07DACBD9AAD463A12BE5CE72

Name: Keerthana.M

Mail Id: keerthanamoorthy20072006@gmail.com

Nm Id: 31F65E4007E7AE2B51F419FA4642615A

Name: Janapriya.J

Mail Id: janapriya6002@gmail.com

Nm Id: E40873D485834048258410B873E64B76

Name: Janani.S

Mail Id: janani2511jannu@gmail.com

Nm Id: F7CEF453D877815B19C44B95F8F5DD53

2. PROJECT OVERVIEW:

AI WeatherWise API is a RESTful backend application developed using Node.js and Express.js to provide real-time weather information and intelligent weather insights. It uses MongoDB and Mongoose for secure data storage and management. The system integrates Google Gemini AI to generate weather summaries and personalized recommendations based on current weather conditions. It also provides secure JWT authentication and allows users to manage their favorite locations. The APIs can be tested using Postman.

KEY OBJECTIVES:

- Provide real-time weather information for selected locations.
- Generate clear weather summaries using Google Gemini AI.
- Provide personalized recommendations based on weather conditions.
- Implement secure user registration and login.
- Store and manage user and location details efficiently.
- Allow users to maintain their favorite weather locations.
- Develop a modular and maintainable RESTful backend system.

KEY FEATURES:

1. **User Registration** – Allows new users to create an account.
2. **User Login** – Provides authenticated access using JWT.
3. **Weather Information** – Retrieves current weather details for locations.
4. **AI Weather Summary** – Converts weather data into understandable summaries.
5. **AI Recommendations** – Provides suggestions based on current weather conditions.
6. **Favorite Locations** – Enables users to save and manage preferred locations.
7. **API Testing** – Supports testing of REST API endpoints using Postman.

3. SYSTEM REQUIREMENTS:

The System Requirements define the software and hardware resources required to develop, configure, and run the AI WeatherWise API. These requirements ensure that the backend, database, AI services, and API testing tools can work properly.

Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Node.js:** Version 16 or above for running the backend.
- **npm:** Version 8 or above for managing dependencies.
- **Express.js:** For developing RESTful APIs.
- **MongoDB & Mongoose:** For database storage and management.
- **Google Gemini AI:** For weather summaries and recommendations.
- **Postman:** For API testing.
- **Visual Studio Code:** For project development.

Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** Minimum 8 GB.
- **Storage:** At least 1 GB of available disk space.

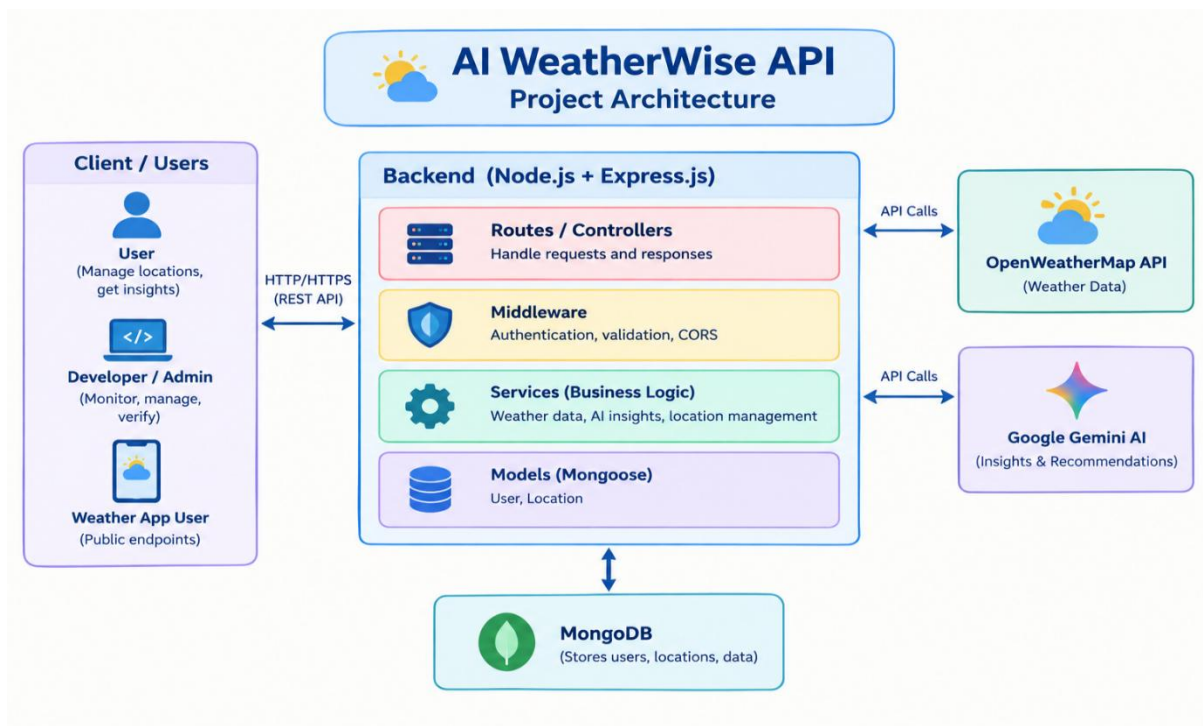
4. PROJECT ARCHITECTURE:

The AI WeatherWise API follows a modular architecture using Node.js and Express.js. It separates routes, controllers, services, middleware, and models for better management. MongoDB stores user and location data, while Gemini AI provides weather summaries and recommendations.

Main Architecture Components:

1. **Client / Postman** – Sends requests to the API.
2. **Express.js Server** – Receives and processes API requests.

3. **Routes** – Direct requests to the appropriate controllers.
4. **Controllers** – Handle the main application operations.
5. **Services** – Manage weather data and Gemini AI processing.
6. **Middleware** – Handles authentication and request validation.
7. **MongoDB + Mongoose** – Stores and manages user and location data.
8. **Google Gemini AI** – Generates weather summaries and recommendations.



5. ENTITY - RELATIONSHIP (ER) DIAGRAM DESCRIPTION :

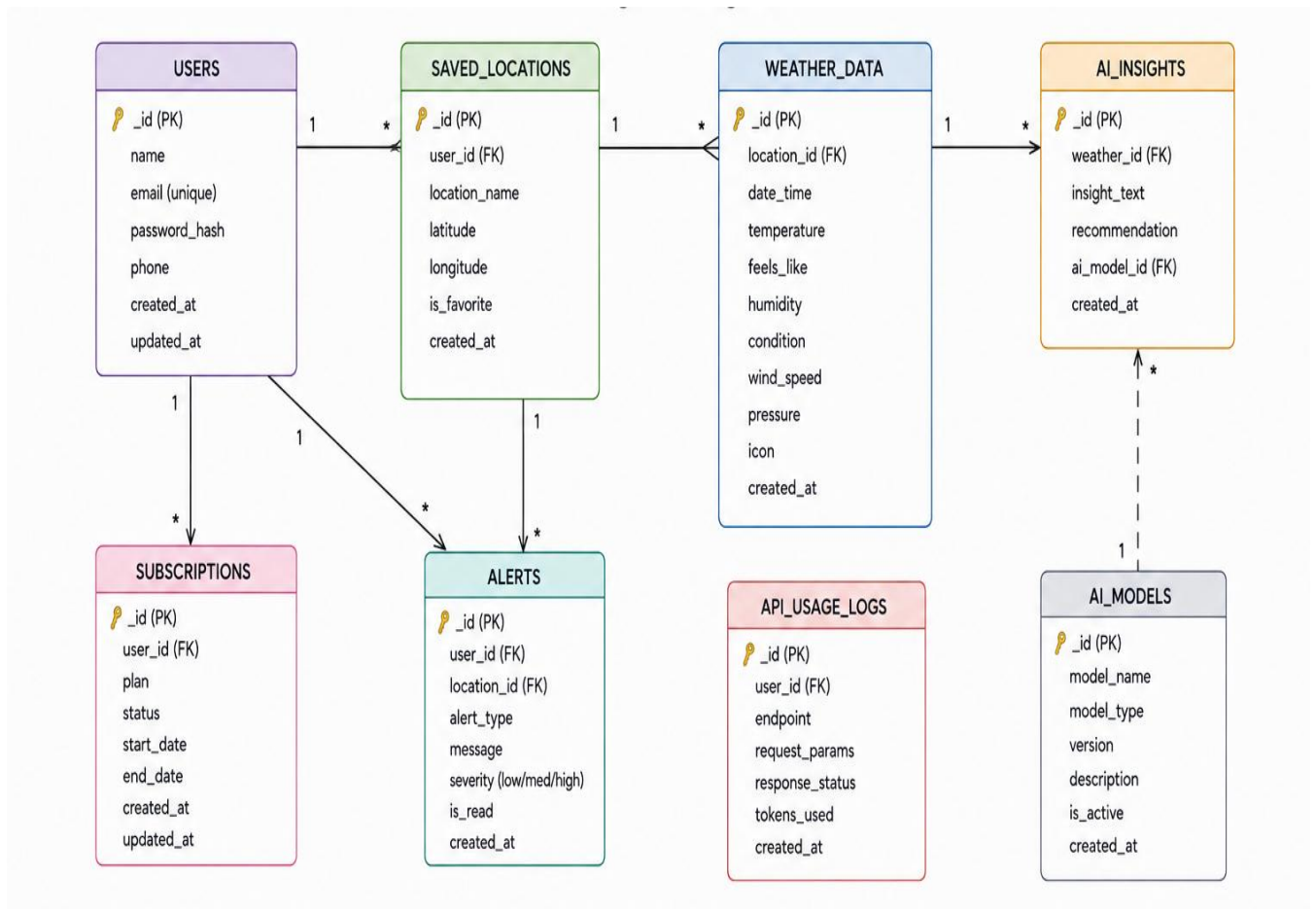
The data architecture manages User profiles and their associated favorite Locations through logical references:

1. User Entity

- **_id** (Primary Key - ObjectId)
- **name** (String, Required)
- **email** (String, Required, Unique)
- **password** (String, Hashed, Required)
- **role** (String, Default: "reader")

2. Location Entity

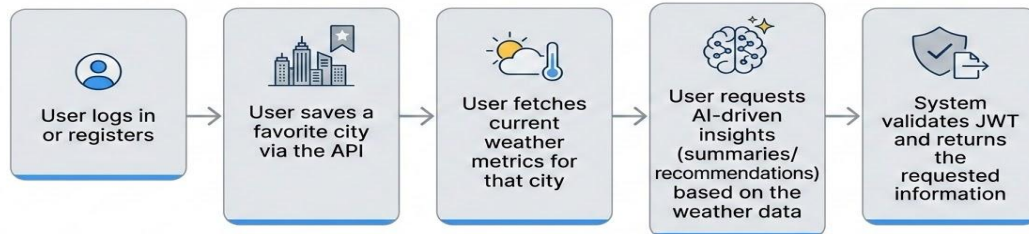
- `_id` (Primary Key - ObjectId)
- `user` (Foreign Key referencing User Collection)
- `city` (String, Required)
- `country` (String, Required)
- `createdAt` (Date, Default: Now)



User Flow:

1. User logs in or registers.
2. User saves a favorite city via the API.
3. User fetches current weather metrics for that city.
4. User requests AI-driven insights (summaries/recommendations) based on the weather data.
5. System validates JWT and returns the requested information.

AI WeatherWise API User Workflow



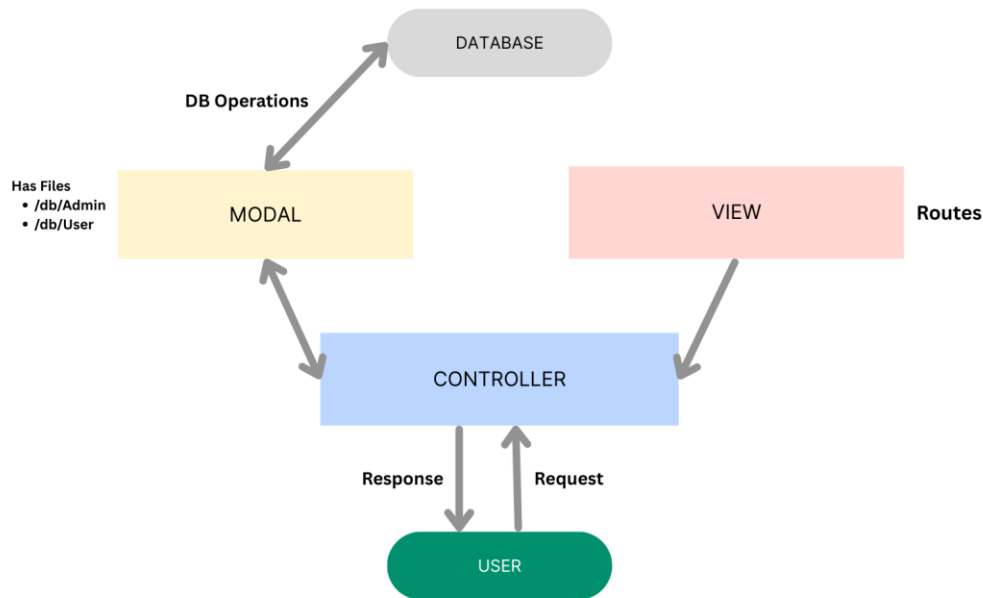
Roles and Responsibilities:

- **Admin:**
 - Responsible for user account management, including creating, deleting, or suspending accounts.
 - Oversight of API health and system logs to monitor performance.
 - Monitoring and managing database integrity.
- **Registered User:**
 - Responsible for creating and maintaining a list of favorite locations.
 - Initiating requests for real-time weather data.
 - Interacting with the Gemini AI service for personalized weather summaries and activity recommendations.
 - Managing their own authentication credentials, such as updating profile details.

6. MVC ARCHITECTURE PATTERN:

The AI WeatherWise API follows the Model-View-Controller (MVC) architecture pattern to separate data management, application logic, and API routing. This separation makes the system easier to develop, maintain, and manage.

- ❑ **Model:** Manages the structure and storage of user and location data in MongoDB.
- ❑ **Controller:** Processes API requests, handles application logic, and returns JSON responses.
- ❑ **View:** In this backend API, the routing layer acts as the interface between client requests and controller functions.



Model Layer (Data Blueprint):

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix):

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

View Layer (Routing Network Interface):

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API **Routing Layer**. It links network requests on specified paths directly to their associated controller handler logic maps.

7. PROJECT SETUP AND CONFIGURATION:

Creating the Project Folder

- Create a new folder named **AI Weatherwise**
- Open the project folder in **Visual Studio Code**.
- Initialize the project using Node.js and install the required dependencies.

Project Initialization:

Open the project folder in the VS Code terminal and execute:

npm init -y

Install the required dependencies:

npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer@google/genai

Install Nodemon for development:

npm install --save-dev nodemon

Project Setup:

AI-WeatherWise/

|

|— src/

|

| |— models/

|

| |— controllers/

|

| |— routes/

|

| |— services/

|

| |— middleware/

|

| |— config/

|

|— .env

|

|— .gitignore

|

|— package.json

|

|— package-lock.json

|

|— index.js

|

|— postman_collection.json

|

|— README.md

Environment Configuration:

Create a **.env** file in the project root and configure the required environment variables.

PORT=5000

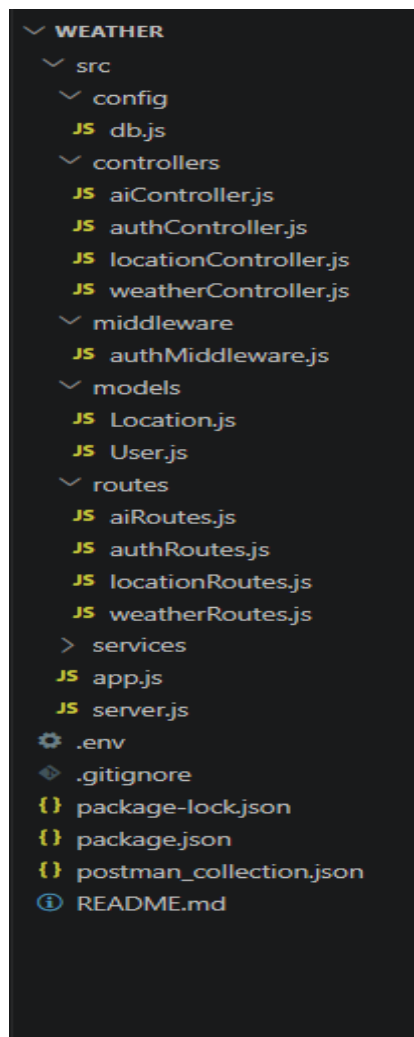
MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret

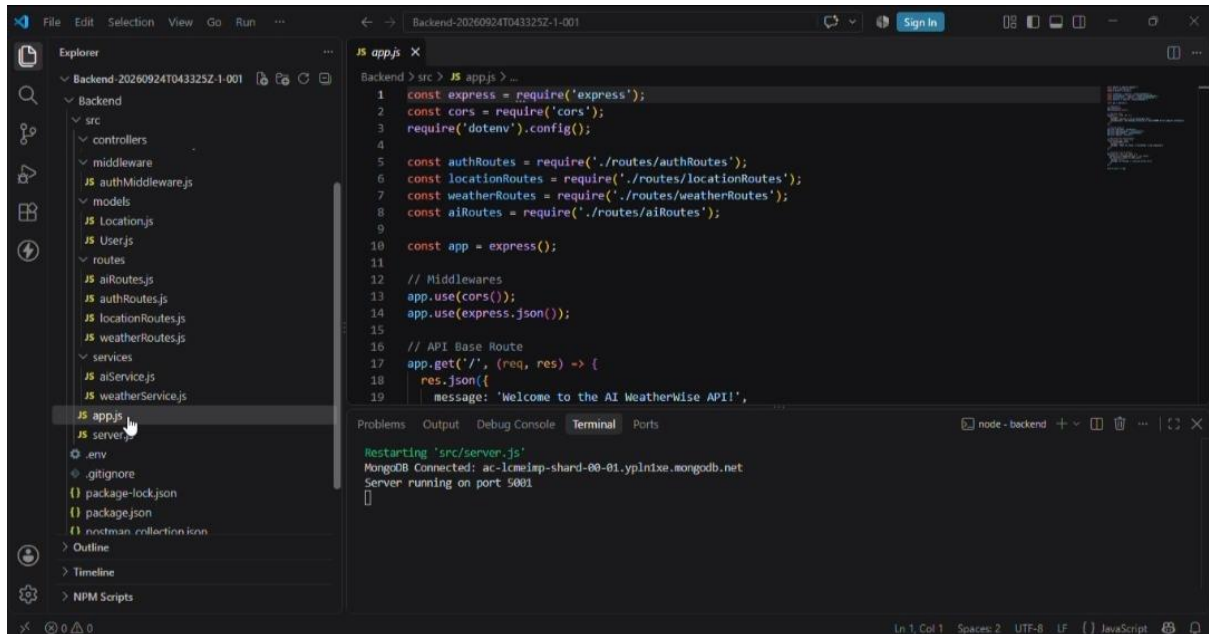
GEMINI_API_KEY=your_gemini_api_key

8. BACKEND DEVELOPMENT:

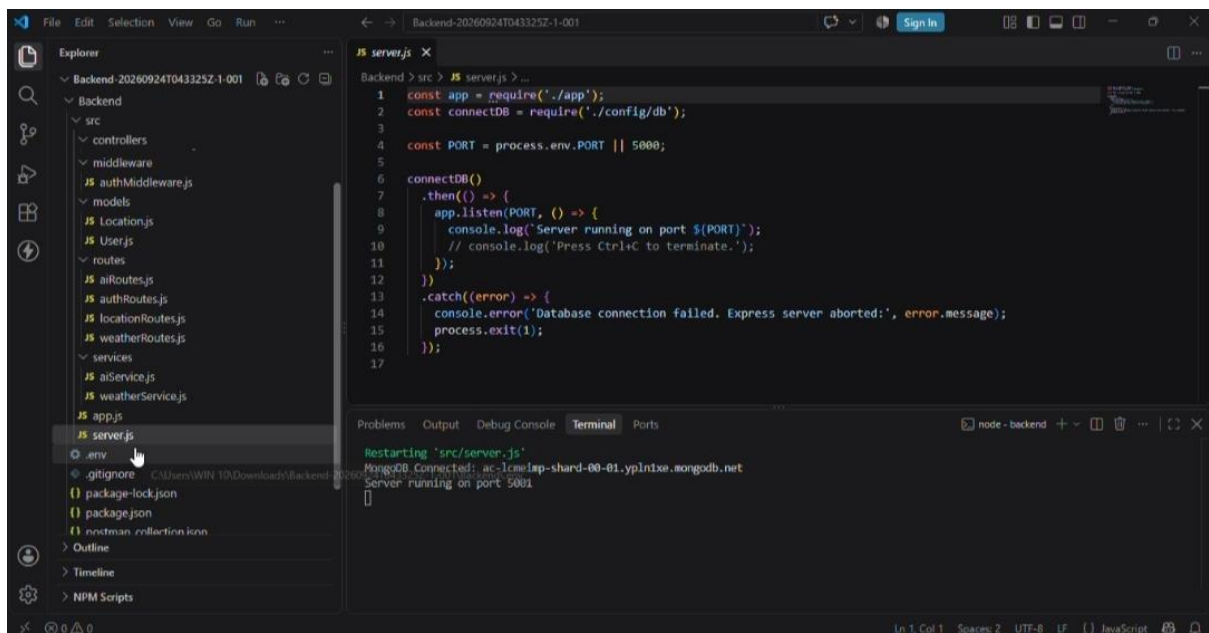
The AI weatherwise backend is developed using **Node.js**, **Express.js**, and **MongoDB**, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.



App.js:



Server.js:



9. DATABASE CONFIGURATION:

1.Configure MongoDB

- Install Mongoose using `npm install mongoose`.
- Create the MongoDB database connection.
- Define the required Mongoose schemas and models.
- Establish the database connection before performing backend operations.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

Location Schema:

Defines the structure for storing users' favorite location details such as city, country, and user reference.

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

User Schema:

The User Schema defines the structure of user data stored in MongoDB. It contains details such as name, email, password, and role. It helps manage user registration and authentication data.

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

Database Connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

10.PROJECT EXECUTIONS:

The AI weatherwise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

Running the Backend Server:

- Install all required dependencies:

npm install

- Start the development server:

npm start

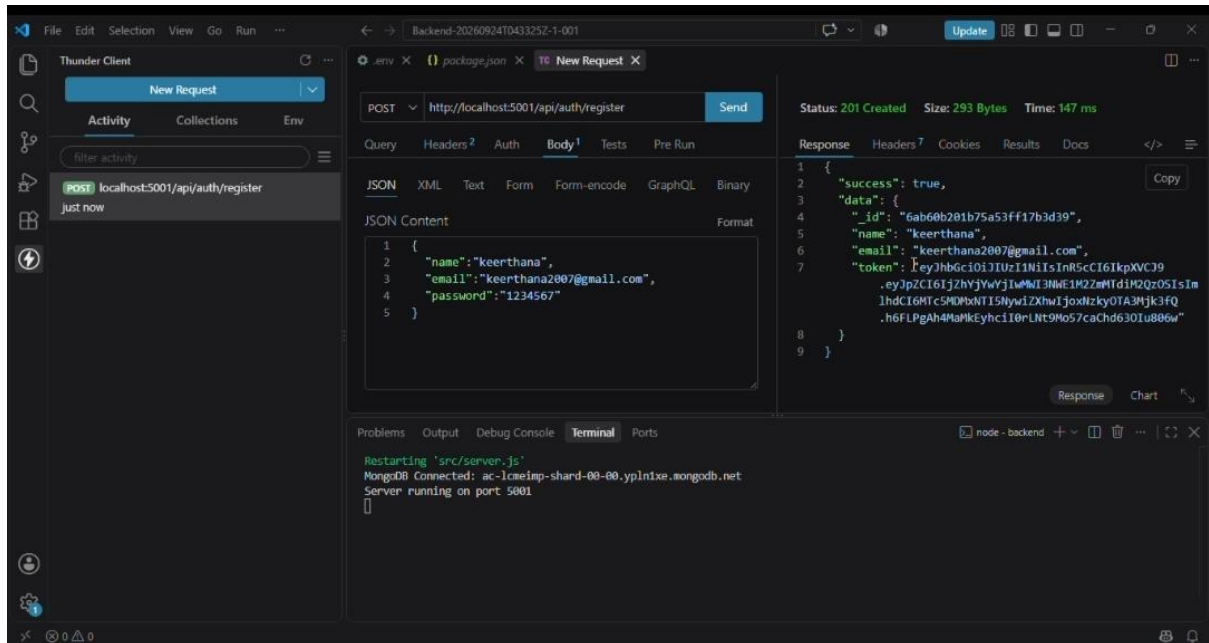
- ✓ After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API Testing:

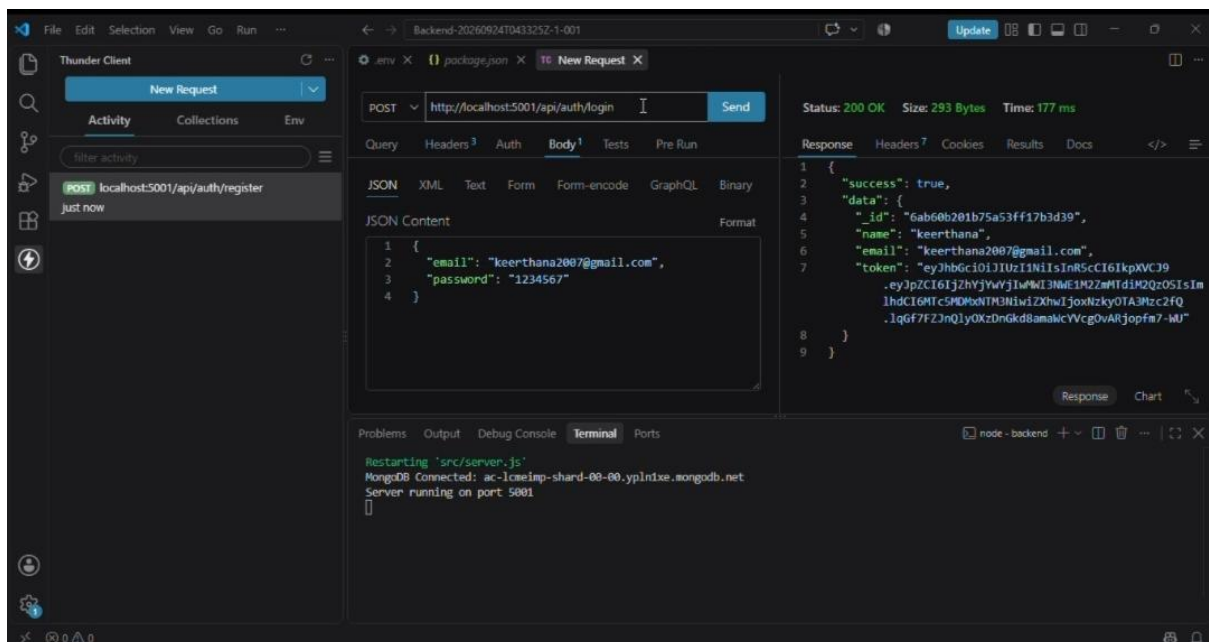
API testing is performed using Postman to verify the functionality and responses of the AI WeatherWise APIs.

- **User Registration** – Tests the creation of a new user account.
- **User Login** – Tests user authentication and login access.
- **Fetch Weather Data** – Tests retrieving current weather data for Chennai.
- **Generate Weather Summary** – Tests generating a clear summary of the weather data.
- **Generate Weather Recommendation** – Tests generating recommendations based on the weather conditions.

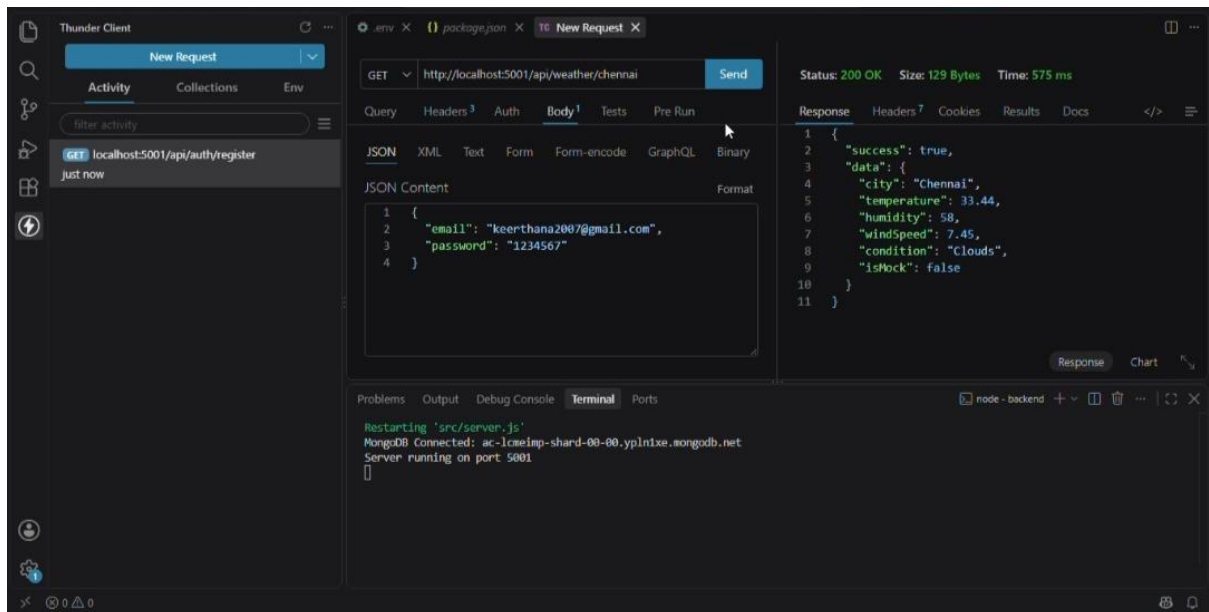
USER REGISTRATION: POST: <http://localhost:5001/api/auth/register>



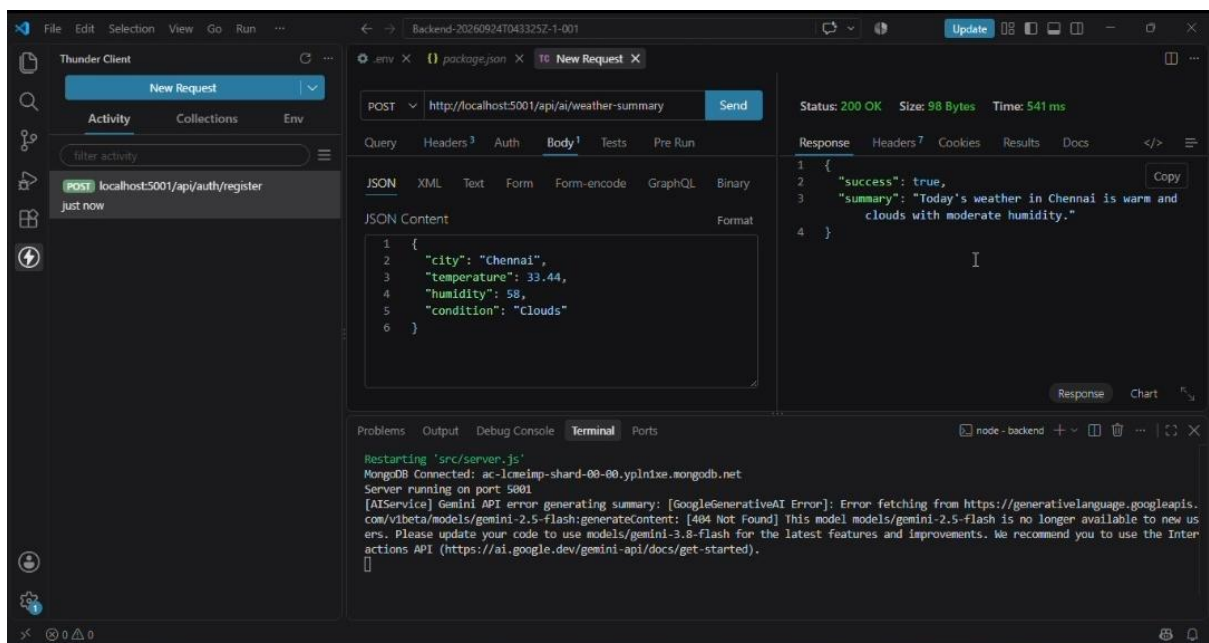
USER LOGIN: POST: <http://localhost:5001/api/auth/login>



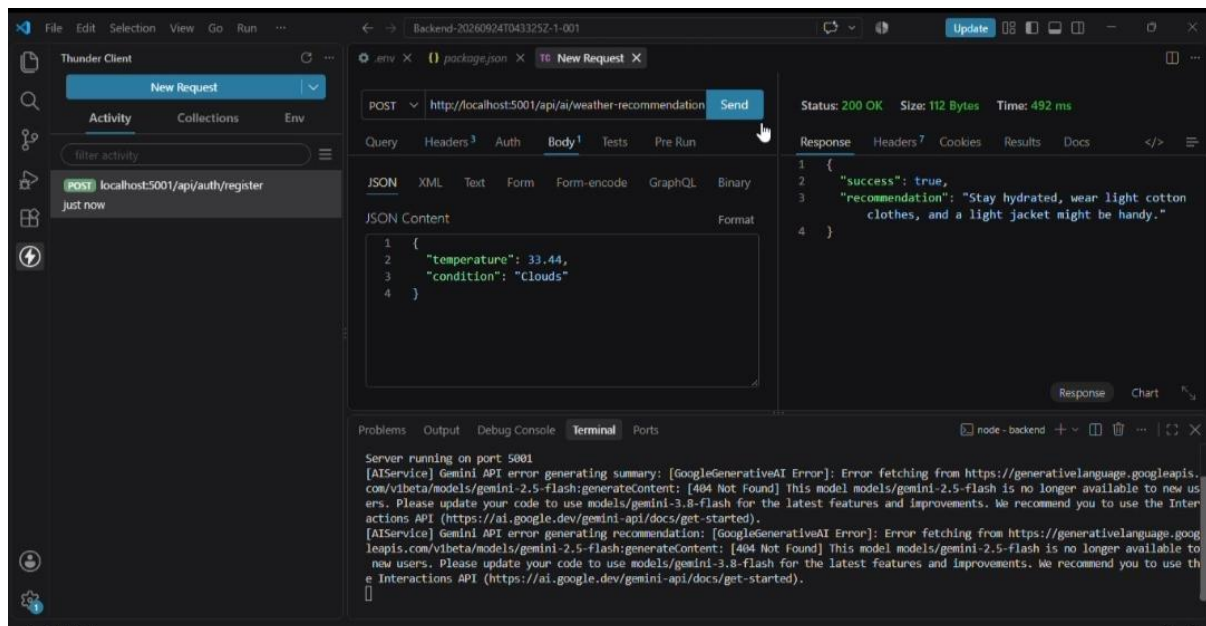
FETCH WEATHER DATA: GET: <http://localhost:5001/api/weather/chennai>



GENERATE WEATHER SUMMARY: POST: <http://localhost:5001/api/ai/weather-summary>



GENERATE WEATHER RECOMMENDATION: POST: <http://localhost:5001/api/ai/weather-recommendation>



Testing Results:

The AI WeatherWise API was tested using Postman to verify the main API functions. The User Registration, User Login, Fetch Weather Data, Generate Weather Summary, and Generate Weather Recommendation APIs were checked for proper responses. The testing confirmed that the APIs perform their intended functions correctly.

11. CONCLUSION:

The **AI WeatherWise API** provides real-time weather information through a secure backend system. It uses **Node.js**, **Express.js**, **MongoDB**, and **Gemini AI** for processing and data management. The system provides weather information, summaries, and recommendations based on current conditions. User registration and login provide secure access to the APIs. The main APIs were tested using **Postman** to verify their functionality. Overall, the project provides useful and understandable weather insights to users.

12. FUTURE ENHANCEMENTS:

- Add support for weather information from multiple cities and locations.
- Improve AI-generated weather summaries and recommendations.
- Add weather alerts and notifications for users.
- Develop a user-friendly frontend application.
- Add more personalized recommendations based on user preferences.
- Improve API performance, security, and scalability.

13. PROJECT DEMO:

API Testing Video: API LINK : https://drive.google.com/file/d/1xTd6-m4UaqqXTbmW2RaQvzMjGqmP-ZNM/view?usp=drive_link

Demo Video Link: DEMO VIDEO :
https://drive.google.com/file/d/1HtCEfjuxj7Pj32kc2QTYqWnals5eEoI6/view?usp=drive_link